

External Practical

Github: https://github.com/Madhav-Gohel/loyalty_card_storage

Practical No. 2

Aim:

Problem Statement:

At **Everyday Rewards Inc.**, our customers currently juggle dozens of physical loyalty cards in their wallets or phone wallets, causing delays at checkout, missed rewards opportunities, and frustration when cards expire unnoticed. To enhance customer experience and drive repeat visits, we need a **Flutter-based Loyalty Card Storage App** that digitises card management, streamlines redemption at point-of-sale, and keeps users informed about their rewards and expiration dates in real time. The app must let users quickly scan or manually enter card details, display barcodes/QR codes on demand, and trigger push notifications for new offers or impending expirations. Additionally, it should work offline—allowing users to retrieve and present cards without connectivity—and sync seamlessly with a cloud backend once back online to back up data and fetch updates. The solution should be intuitive for end users, performant even with hundreds of stored cards, and secure enough to protect personal account information. Your task is to develop a functional prototype that addresses these needs while adhering to best practices in Flutter development, state management, and data persistence.

Key Pain Points to Solve:

- Physical card bulk and wallet clutter leading to forgotten or lost cards.
 - Checkout delays when users must locate and scan multiple cards.
 - Missed rewards or points expiration due to lack of timely reminders.
 - Inability to access card barcodes when offline (e.g., in basements, on planes).
- The ideal solution will balance performance, usability, and reliability while demonstrating clean, maintainable code.**

Note:

- **Offline Access:** Cache all card data (including images/barcodes) locally for instant retrieval.
- **Sync Logic:** On network reconnection, reconcile local changes (new cards added, cards used) with the cloud backend.
- **Notifications:** Real-time push for reward milestones and expiration alerts.
- **Data Security:** Encrypt stored card numbers and API communications.

Provide Version Control using GitHub.

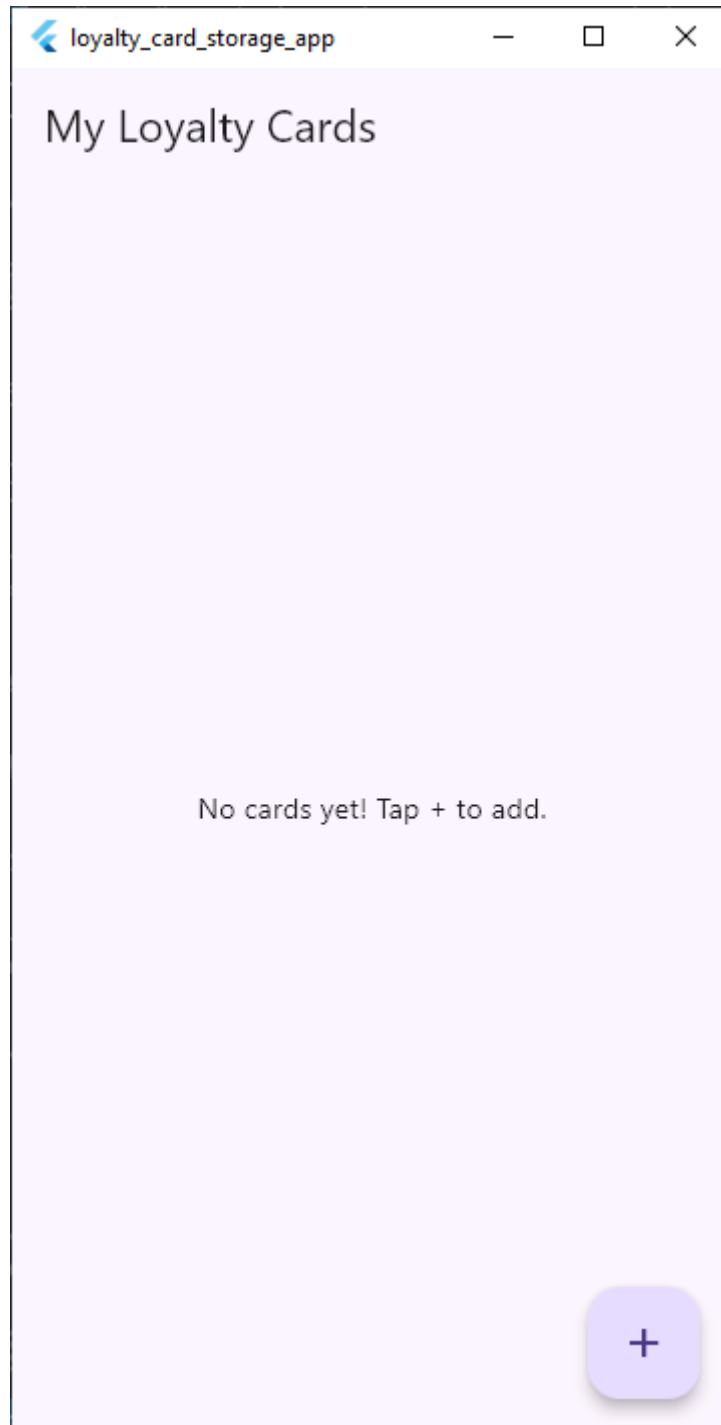
Repository has at least four commits.

- Commit I: Initialization
- Commit II: ...
- Commit III: ...

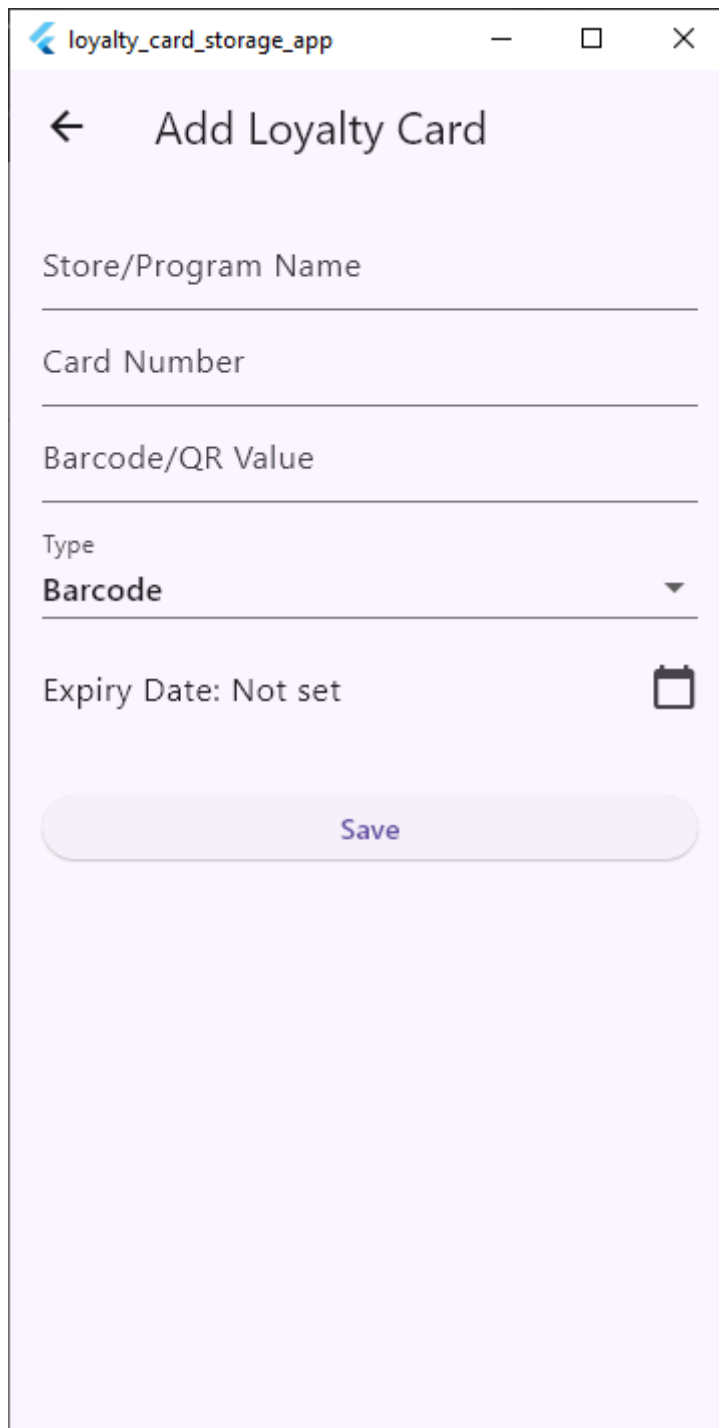
- Commit IV: ...
- ...

Document to Upload:

- PDF Name: StudentID_ProblemStatement2
- List of Screen Shots of Each UI Screen.
- Future Implementation & Conclusion.

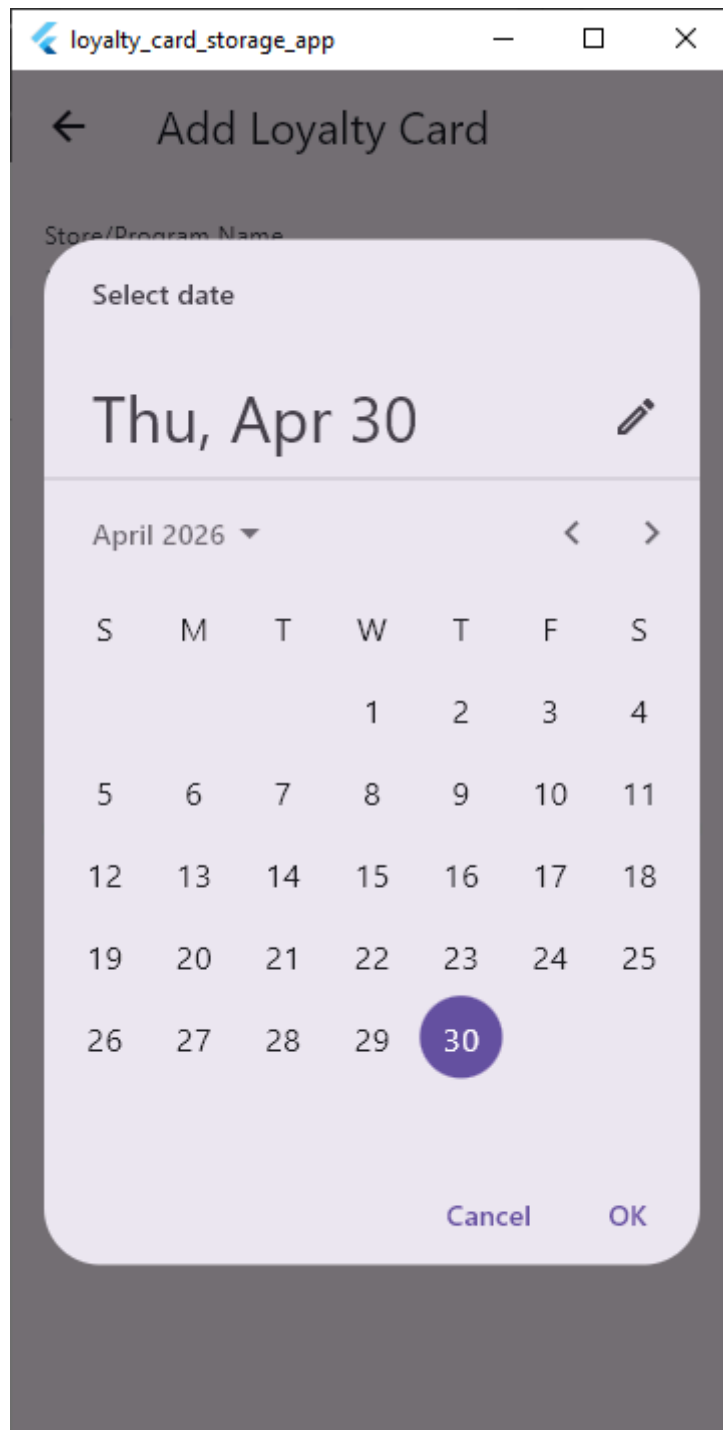
List of Screenshots

Home Screen

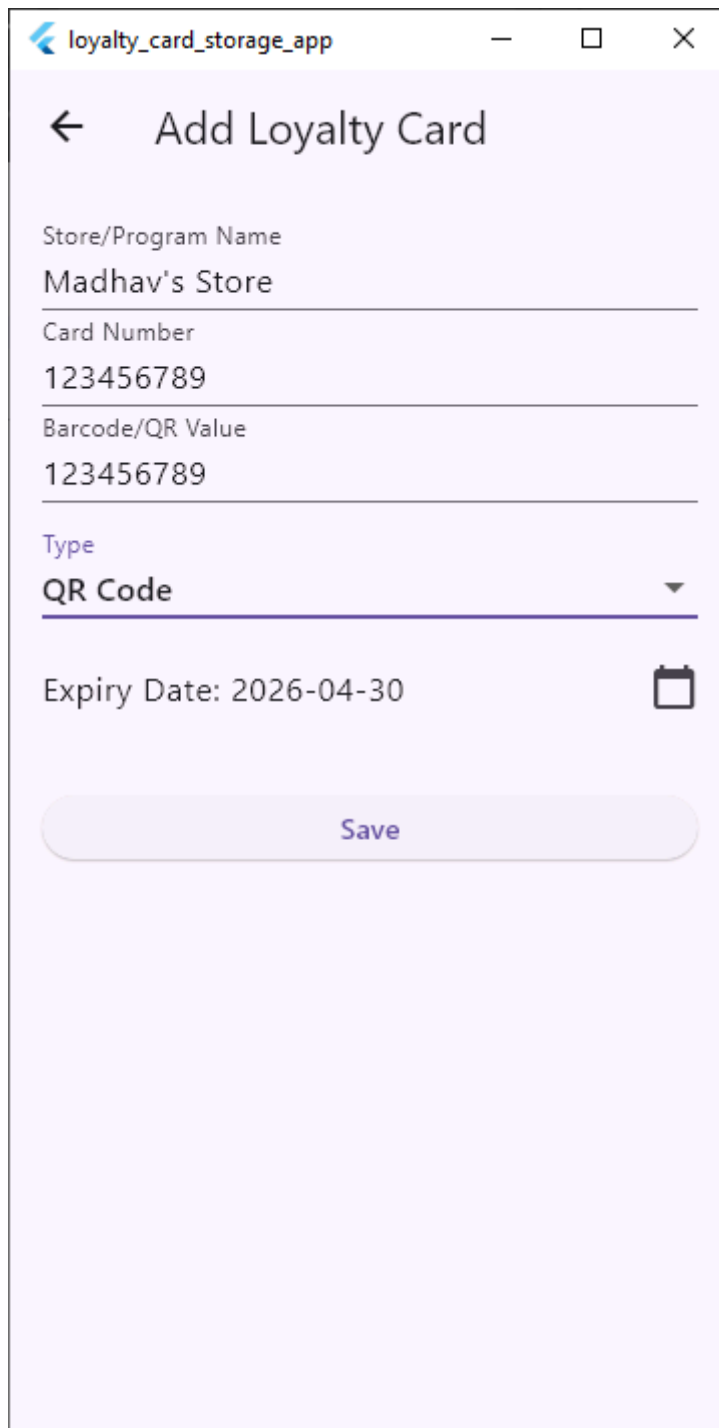


The screenshot shows a mobile application window titled 'loyalty_card_storage_app'. The main heading is 'Add Loyalty Card' with a back arrow on the left. Below the heading are five input fields: 'Store/Program Name', 'Card Number', 'Barcode/QR Value', 'Type' (a dropdown menu currently showing 'Barcode'), and 'Expiry Date: Not set' (with a calendar icon to its right). At the bottom of the form is a large, rounded 'Save' button.

Add card



Expiration date

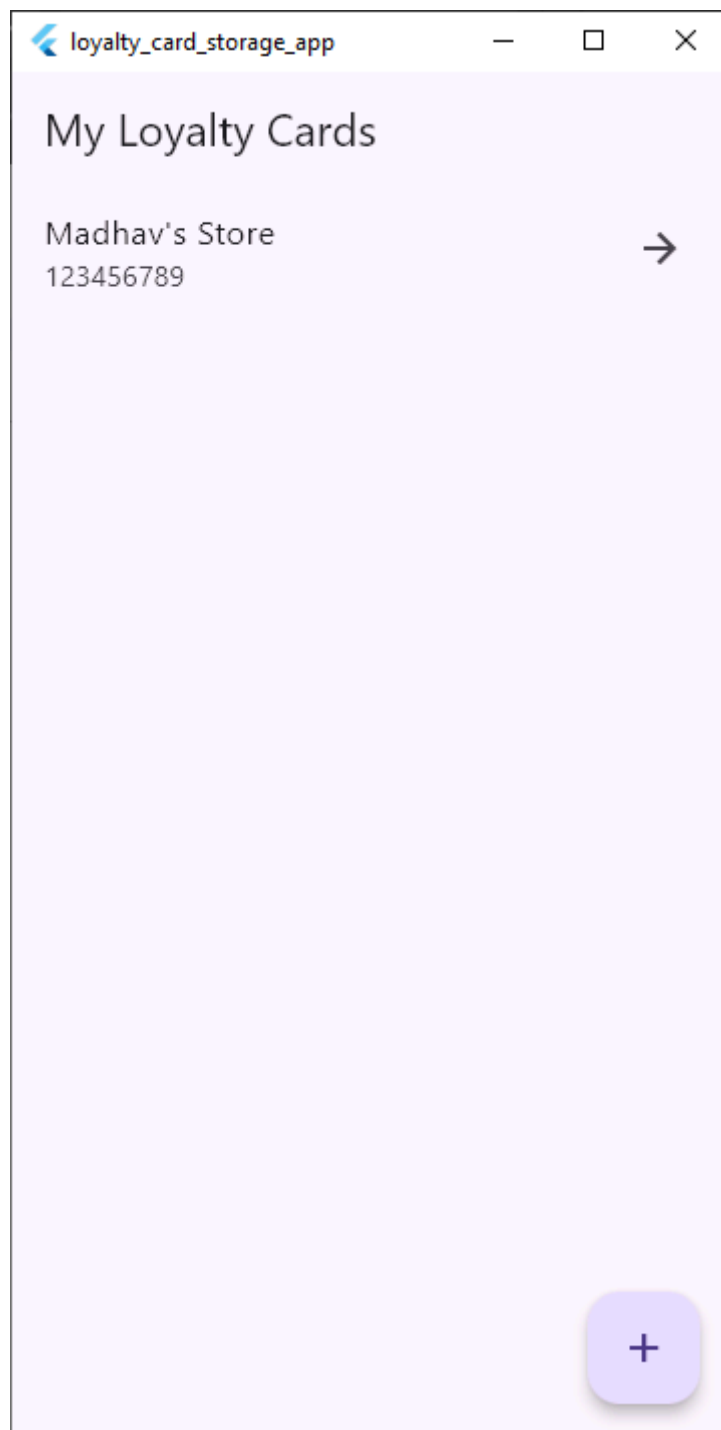


The screenshot shows a mobile application window titled 'loyalty_card_storage_app'. The main heading is 'Add Loyalty Card' with a back arrow on the left. The form contains the following fields:

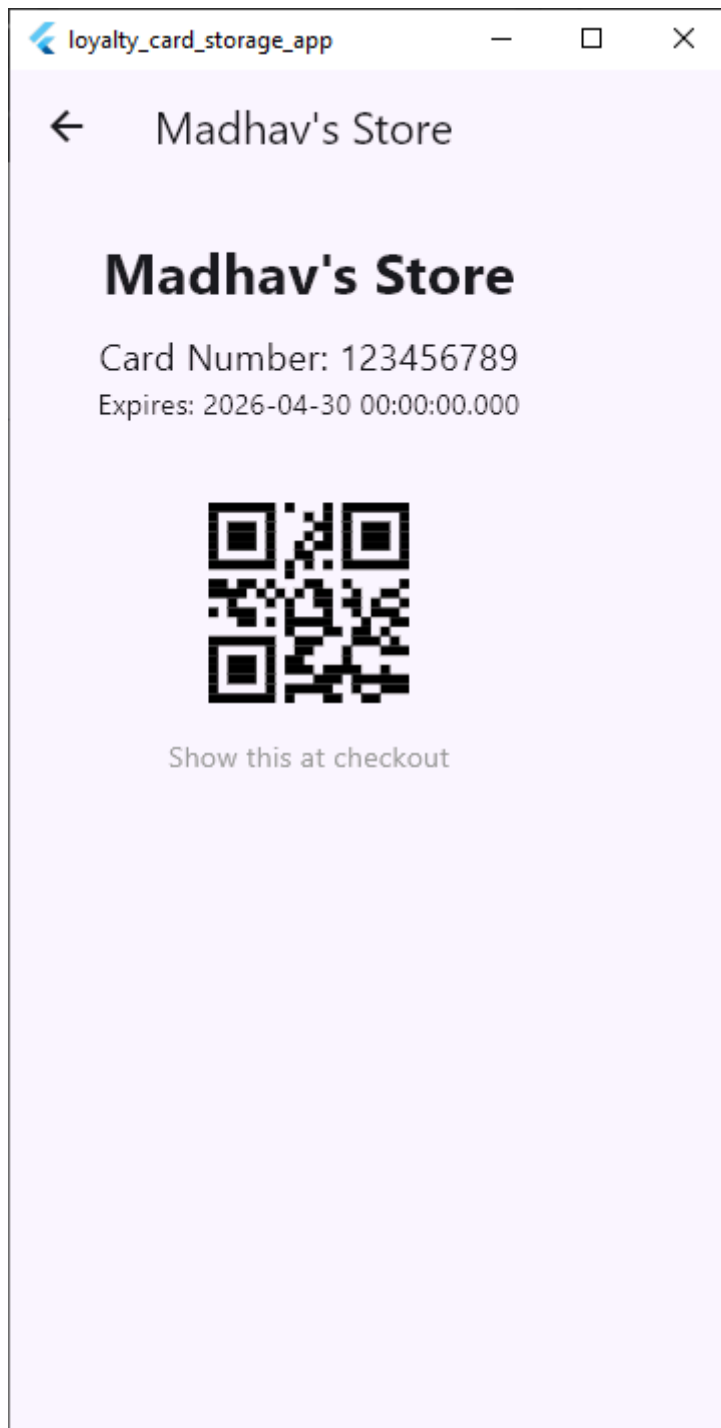
- Store/Program Name:** Madhav's Store
- Card Number:** 123456789
- Barcode/QR Value:** 123456789
- Type:** QR Code (indicated by a dropdown arrow)
- Expiry Date:** 2026-04-30 (with a calendar icon)

A 'Save' button is located at the bottom of the form.

Added all card details



Add to the list



Reward Details

Future Implementation:

In the future, the application can be enhanced by integrating cloud-based push notification services like Firebase Cloud Messaging (FCM) to enable cross-device notifications. This would allow users to receive reminders about card expirations even when the app is closed or running in the background. Additionally, implementing advanced scheduling algorithms with periodic checks can ensure notifications are sent reliably for long-term usage. Features such as user-configurable notification preferences, multi-language support for notifications, and integration with wearable devices (e.g., smartwatches) can further improve user experience.

Conclusion:

The addition of push notifications for expiring loyalty cards significantly enhances the app's functionality by ensuring users are reminded to utilize their cards before expiration. Leveraging local notifications, the current implementation provides a reliable, offline-capable solution that aligns with the app's existing architecture. This feature not only improves user engagement but also demonstrates the app's commitment to delivering a seamless and efficient experience for managing loyalty cards.